Mock Exam B — Weeks 1 to 7

DSA27 · Data Structures and Algorithms · midterm practice · 60 marks: Section A 30 · Section B 30

Question bank practice paper

Fall 2026

Contents

Section A — Multiple choice (30 marks) **1**

Section B — Written answers (30 marks) **7**

After the exam **8**

> ***How to use this paper.*** *Give yourself **two hours**, closed book, no computer, and answer on paper. Every question is printed here in full, so you need nothing else. It covers all seven lectures before the midterm, with three to five questions from each in Section A. The questions were added to the bank for the midterm, and **none** of them appears on the other three mock papers, so you can sit this one after the first weeks 1–7 mock. Mark yourself with the answers **only after** you have finished.*
>
> *For a different paper every time, with a timer and a report by lecture, use the online midterm simulator. The midterm's own format and date are announced in the lecture and on the WhatsApp channel.*

Section A — Multiple choice (30 marks)

One mark each. Choose **one** answer out of four and write the letter on your answer sheet. The tag after each number is the level of the question: [what] asks you to recall, [how] to work it out, [why] to explain.

Lecture 1 — Python and the course

A1. [how] What is `f'{7:03d}|{3.14159:.2f}'`?

- **a)** `'7|3.14'`
- **b)** `'007|3.14'`
- **c)** `'700|3.141'`
- **d)** `'007|3.1'`

A2. [how] An $O(n^2)$ program takes 2 seconds on today's data. The data grows **ten times** larger. About how long will it take now?

- **a)** 20 seconds
- **b)** 40 seconds
- **c)** 200 seconds
- **d)** 4 seconds

A3. [why] A shop stores prices in pounds as `float` values and adds them up. Why does Lab 01 say not to, and what should it do instead?

- **a)** floats are slower than ints; it should use strings
- **b)** floats cannot hold negative values; it should use `abs()`
- **c)** floats overflow above a million; it should use smaller units
- **d)** amounts like 0.10 cannot be stored exactly in binary, so totals drift; it should store whole piasters as `int`

Lecture 2 — Complexity and arrays

A4. [why] Algorithm A does $1000n$ steps; algorithm B does n^2 steps. Which statement is **true**?

- **a)** B does fewer steps for n < 1,000; A does fewer for every n > 1,000
- **b)** A does fewer steps for every n, because $O(n)$ beats $O(n^2)$
- **c)** B does fewer steps for every n, because its constant is 1
- **d)** They are equally good, because both are polynomials

A5. [how] What is the running time of this loop?

```
i = 0
while i * i < n:
    i += 1
```

- **a)** $\Theta(\log n)$
- **b)** $\Theta(\sqrt{n})$
- **c)** $\Theta(n)$
- **d)** $\Theta(n^2)$

A6. [why] `remove_at` ends with `arr[size - 1] = None`. Why?

- **a)** So that `len(arr)` goes down by one
- **b)** Without it, the shifted values would end up in the wrong order
- **c)** That slot is no longer in use; clearing it removes a stale copy of the last value, which would otherwise keep that object alive
- **d)** The course `Array` raises an error if a slot is not `None`

A7. [what] In C, after `int a[6];`, the statement `a[6] = 0;`:

- **a)** raises an index error at run time
- **b)** makes the array grow to 7 slots
- **c)** is the same as `a[0] = 0`, because indices wrap round
- **d)** writes past the end with no check, over whatever is stored next — a buffer overflow

Lecture 3 — Recursion

A8. [how] With the course `gcd` (`return a` if `b == 0`, else `return gcd(b, a % b)`), how many calls does `gcd(48, 18)` make in total, counting the first?

- **a)** 2
- **b)** 3
- **c)** 18
- **d)** 4

A9. [what] With the course's fast `power` (one call on `exponent // 2`, base case `exponent == 0`), at most how many frames of `power` are on the call stack at once during `power(2, 1000)`?

- **a)** 11
- **b)** 1,000
- **c)** 2
- **d)** 500

A10. [why] `merge_sorted` compares the heads with `left[0] <= right[0]`. What would change if it used `<` instead?

- **a)** equal values would be lost from the result
- **b)** the merge would become $O(n^2)$
- **c)** on a tie the head of `right` would go first, so the merge would no longer be stable
- **d)** it would raise an error when two heads are equal

A11. [how] The course `subsets`:

```
def subsets(items):
    if not items:
        return [[]]
    rest = subsets(items[1:])
    return rest + [[items[0]] + subset for subset in rest]
```

What does `subsets([1, 2])` return, in this order?

- **a)** `[[], [1], [2], [1, 2]]`
- **b)** `[[], [2], [1], [1, 2]]`
- **c)** `[[1, 2], [1], [2], []]`
- **d)** `[[], [1, 2]]`

Lecture 4 — Dynamic arrays

A12. [how] With the course `DynamicArray` (capacity 1, doubling), what are `len(a)` and `a._capacity` after `a = DynamicArray(range(6))`?

- **a)** 6 and 6
- **b)** 8 and 8
- **c)** 6 and 8
- **d)** 6 and 12

A13. [why] Why does `__getitem__` check `index < self._size` rather than `index < self._capacity`?

- **a)** The slots beyond `size` hold nothing the user stored, so reading one must raise `IndexError`, as it does for a Python list
- **b)** The slots beyond `size` do not exist in the `Array` yet
- **c)** Comparing with `size` is faster than comparing with `capacity`
- **d)** It is only needed so that negative indices work

A14. [how] After 9 appends and then 8 calls of `pop()`, what is the capacity of the course `DynamicArray`?

- **a)** 1 — popping gives the memory back
- **b)** 16 — the course pop never shrinks the array
- **c)** 2 — it halves whenever it is a quarter full
- **d)** 4 — it halves whenever it is half full

A15. [why] An array that grows by a constant **+10** slots copies 798,400 elements for 4,000 appends. For 8,000 appends it copies about:

- **a)** the same number — the step is the same
- **b)** twice as many
- **c)** 13 more
- **d)** four times as many

Lecture 5 — Linked lists

A16. [what] `ll = LinkedList([1, 2, 3])`. Which call raises `IndexError`?

- **a)** `ll[3]`
- **b)** `ll.insert_at(3, 9)`
- **c)** `ll[-1]`
- **d)** `ll.insert_at(0, 9)`

A17. [how] In a doubly linked list, `x` is a node that is neither the first nor the last. Which two assignments remove it from the list?

- **a)** `x.next = x.prev` and `x.prev = x.next`
- **b)** `x.prev = None` and `x.next = None`
- **c)** `x.prev.next = x.next` and `x.next.prev = x.prev`
- **d)** `x.prev.next = x.prev` and `x.next.prev = x.next`

A18. [why] A student adds `_tail` to the list. Their `append` decides "empty" by testing `self._tail is None`, and their `pop_front` updates `head` and `_size` but never `_tail`. What does this print?

```
ll = LinkedList([7])
ll.pop_front()
ll.append(5)
print(list(ll), len(ll))
```

- **a)** `[] 1`
- **b)** `[5] 1`
- **c)** `[7, 5] 2`
- **d)** it raises `AttributeError`

A19. [what] `LinkedList().pop_front()`:

- **a)** returns `None`
- **b)** returns 0
- **c)** raises `AttributeError`
- **d)** raises `IndexError`

A20. [why] You hold a reference node to a node in the middle of a long **singly** linked list. Inserting a new node immediately **before** it costs:

- **a)** $O(1)$, like inserting after it
- **b)** amortised $O(1)$
- **c)** $O(n)$: the node before it must be found by walking from the head
- **d)** $O(\log n)$

Lecture 6 — Stacks

A21. [how] The shunting-yard algorithm converts `2 * 3 + 4` to:

- **a)** `2 3 4 + *`
- **b)** `2 3 * 4 +`
- **c)** `2 * 3 4 +`
- **d)** `+ * 2 3 4`

A22. [how] `evaluate_postfix(["6", "2", "/", "3", "*"])` returns:

- **a)** 1.0
- **b)** 18.0
- **c)** 9.0
- **d)** it raises `ValueError`

A23. [why] A version of `is_balanced` ends with `return True` instead of `return openers.is_empty()`. Which input does it get **wrong**?

- **a)** `([)]`
- **b)** `)(`
- **c)** `(a))`
- **d)** `((a)`

A24. [what] `s = Stack([1, 2, 3])`. What does `s.pop()` return?

- **a)** 1
- **b)** 3
- **c)** `[1, 2, 3]`
- **d)** it raises `IndexError`

A25. [why] Two `peek()` calls in a row, with nothing in between, return two **different** values. The most likely bug is that peek:

- **a)** reads index 0 instead of the last slot
- **b)** forgets to raise `IndexError` on an empty stack
- **c)** is $O(n)$ instead of $O(1)$
- **d)** removes the top, as pop does

Lecture 7 — Queues

A26. [how] A `CircularQueue(5)`, starting empty: enqueue A, B, C, D, E; dequeue three times; enqueue F, then G. What is in the block, from slot 0 to slot 4?

- **a)** `F G None D E`
- **b)** `D E F G None`
- **c)** `None None None D E F G`
- **d)** `F G C D E`

A27. [how] A full `CircularQueue` of capacity 4 holds C D E F (front to back), with head = 2. `enqueue(G)` makes the correct `_grow` double the block first. Afterwards, head is ___ and G is in slot ___:

- **a)** 2 and 6
- **b)** 0 and 0
- **c)** 2 and 2
- **d)** 0 and 4

A28. [what] A ring buffer that **leaves one slot unused** to tell full from empty has a block of 8 slots. It holds at most:

- **a)** 8 items
- **b)** 7 items
- **c)** 9 items
- **d)** 4 items

A29. [what] A graph search dequeues a node, then enqueues its unvisited neighbours, and repeats. If the queue is replaced by a **stack** (pop instead of dequeue, push instead of enqueue), the same loop:

- **a)** still visits the nodes breadth-first
- **b)** visits the nodes depth-first
- **c)** never stops
- **d)** visits only the start node

A30. [how] A `CircularQueue` with capacity 6 has head = 4 and size = 5. Front to back, the queue's items are in slots:

- **a)** 4, 5, 0, 1, 2
- **b)** 4, 5, 6, 7, 8
- **c)** 0, 1, 2, 3, 4
- **d)** 4, 3, 2, 1, 0

Section B — Written answers (30 marks)

Answer **all** the questions. The marks are shown in brackets; as a guide, spend about two minutes per mark. Write in full sentences, and draw where a drawing helps.

B1. [why] *(3 marks)* "`print` is not `return`." Explain the difference with a two-line function, say what a test sees in each case, and explain why the exercises use parameters and `return` rather than `input()` and `print()`.

B2. [why] *(4 marks)* `values` is a Python list of n numbers. Give the running time of each numbered part of this function, justify each with one of the five rules, and give the total. Why is part (3) **not** $O(n^2)$, although it hides a slice and an `in` inside a loop?

```
def report(values):
    total = 0
    for v in values:                # (1)
        total += v
    i = len(values)
    while i > 1:                    # (2)
        i //= 2
    for v in values:                # (3)
        if v in values[:10]:
            total -= v
    return total
```

B3. [why] *(4 marks)* Three versions of `power(x, n)`: (i) `x * power(x, n - 1)`; (ii) `half = power(x, n // 2)`, then `half * half` (× x if n is odd); (iii) `power(x, n // 2) * power(x, n // 2)` (× x if n is odd). For each, write the recurrence and its solution, and say what happens for n = 4,096.

B4. [how] *(4 marks)* A `DynamicArray` holds `a b c d` in capacity 4. Describe every step of `insert_at(1, "x")`, give the final contents, size and capacity, and count the elements copied and shifted. Why must `insert_at` check for a full array **before** it shifts?

B5. [how] *(4 marks)* In the course `LinkedList`, explain `insert_at(index, value)` step by step: the check on `index`, the case `index == 0`, the walk, and the splice. Why does the walk stop after `index - 1` steps? Give the best-case and worst-case cost, and show what goes wrong if the loop is written `for _ in range(index):`, both for an index in the middle and for `index == len(ll)`.

B6. [why] *(4 marks)* A student's `infix_to_postfix` pops an operator only on **strictly higher** precedence (> instead of >=). Give one input that it still converts correctly and one that it gets wrong, with both postfix results and their values. Explain why the tests can miss this bug, and name an operator for which > is the right rule.

B7. [how] *(3 marks)* Run the whole calculator on `( 8 - 2 ) / ( 1 + 2 ) * 4`: convert it to postfix with the shunting-yard algorithm, then evaluate the postfix with a stack, showing the stack after each operator. Why is the answer a float?

B8. [why] *(4 marks)* Three ring-buffer bugs: the tail computed as `head + size` with no %; the

head moved with `head += 1` with no `%`; and a `_grow` that never updates `_capacity`. For each, say what goes wrong and the shortest operation sequence that shows it. Why do all three pass a test that never wraps the ring?

After the exam

- **Mark Section A** with the table at the top of the answers: one mark per correct letter.
- **Mark Section B** with the marking points, honestly.
- **Add up by lecture.** Each lecture has 3–5 marks in Section A and 3–7 in Section B. Any lecture where you lost more than half: read its handout again, run its interactive lab, and do that week's question bank.
- **Look at the levels too.** Losing [why] questions means you know the steps but not the reasons — explain each step aloud to a friend.